

DevOps

Лекція 2

Частина 1

- Написати bash скрипт автоматичного backup який робить резервне копіювання указаної папки з стисненням у tar.gz формат, timestamp у назві файлу, логуванням процесу та очищенням старих backup (зберігати тільки останні 5)
- Створити bash скрипт моніторингу системи для перевірки використання CPU, RAM або дискового простору з системою алертів у Discord canal при перевищенні порогів

Резервне копіювання

1. Призначення резервного копіювання

- Резервне копіювання (backup) — це процес створення копій даних для відновлення у випадку їх втрати, пошкодження чи видалення. У системному адмініструванні зазвичай використовуються автоматизовані скрипти для регулярного виконання backup.

2. Інструменти для резервного копіювання

- tar — утиліта для архівації файлів і папок.
 - Ключі:
 - -c — створити архів
 - -z — стиснути у форматі gzip
 - -f — вказати ім'я файлу

- -v — показувати процес
 - gzip — утиліта для стиснення (але зазвичай використовується через tar -z).
 - tar -czvf archive.tar.gz /path
- ## 3. Використання timestamp
- Часова мітка (timestamp) дозволяє однозначно ідентифікувати файли backup.
- Формування:
- date +"%Y-%m-%d_%H-%M-%S"
 - tar -czvf project-\$(date +%Y%m%d-%H%M%S).tar.gz project/
 - project-20250924-164532.tar.gz

Резервне копіювання

4. Логування процесу

Логування потрібно для:

- контролю успішності виконання скрипта;
- відстеження помилок.
- Зазвичай використовують перенаправлення виводу:
- `command >> logfile.log 2>&1`
- де:
- `>>` — дописування у файл;
- `2>&1` — перенаправлення `stderr` у `stdout`.

```
LOG_FILE="$BACKUP_DIR/backup.log"
```

```
log() {  
    echo "[$(date '+%Y-%m-%d %H:%M:%S')] $1" | tee -a  
    "$LOG_FILE"  
}
```

```
if [ ! -d "$SOURCE_DIR" ]; then  
    log "❌ ERROR: Directory $SOURCE_DIR doesn't exist!"  
    exit 1  
fi
```

```
log "🚀 The start of process...."
```

5. Очищення старих backup

- Зберігання великої кількості архівів займає місце, тому потрібно видаляти старі.

Можна використати:

- `ls -tp | grep -v '/$' | tail -n +6 | xargs -d '\n' rm --`
або простіше — `find`:
- `find /backup_dir -type f -name "*.tar.gz" | sort | head -n -5 | xargs rm`

Використання ресурсів

CPU моніторинг:

```
CPU_USAGE=$(top -bn1 | grep "Cpu(s)" | \
    sed "s/.*, *\[0-9.\]*\)%* id.*\^1/" | \
    awk '{print 100 - $1}')
```

```
echo "CPU: ${CPU_USAGE}%"
```

- CPU: 56.8%

RAM моніторинг:

```
RAM_USAGE=$(free | grep Mem | \
    awk '{printf("%.0f", ($3/$2) * 100.0)}')
```

```
echo "RAM: ${RAM_USAGE}%"
```

- RAM: 42%

Дисковий простір:

```
df -h | awk '$6 == "/" {
    gsub(/%/,"", $5)
    print $6 ": " $5 "%"
}'
```

- / : 95%

Повідомлення в Discord

```
#!/bin/bash
```

```
THRESHOLD=10
```

```
WEBHOOK_URL="https://discord.com/api/webhooks/1416869719696474132/  
rUDxHHIhiJAqS5mrNw7Mc9kX8sC8apYgCVbB5etu0ScpR4solIDo5zZovLYzR3Nvwyr"
```

```
USAGE=$(df --output=pcent,target / | tail -n1 | awk '{print $1}' | tr -d '%')
```

```
if (( USAGE > THRESHOLD )); then
```

```
    MESSAGE="⚠ Disk Alert! Root partition '/' usage is at ${USAGE}% (threshold: ${THRESHOLD}%)"
```

```
curl -H "Content-Type: application/json" \  
    -X POST \  
    -d '{"content": "${MESSAGE}"' \  
    "$WEBHOOK_URL"
```

```
fi
```

Частина 2

- Розробити Python скрипт для парсингу веб-сервер логів (Apache/Nginx формат) з підрахунком найчастіших IP адрес та User-Agent, аналізом HTTP статус кодів та генерацією HTML звіту. Використати argparse для параметрів командного рядка
- Створити Python скрипт API моніторингу для перевірки доступності endpoints з вимірюванням response time, збереженням результатів у JSON/CSV форматі та інтеграцією з Slack/Discord для notifications при збоях

Парсінг логів в Python

```
import re
```

```
log_pattern = re.compile(  
    r'(?P<ip>\d+\.\d+\.\d+\.\d+)\s'      # IP  
    r'.*\s'  
    r'(?P<time>[^\]]+)\s'              # час  
    r'"(?P<method>\w+)\s(?P<url>\S+)\s(?P<protocol>[^\"]+)"\s' # метод, URL, протокол  
    r'(?P<status>\d{3})\s'             # статус код  
    r'(?P<size>\d+|-)\s'               # розмір  
    r'"(?P<referer>[^\"]*)"\s'         # реферер  
    r'"(?P<user_agent>[^\"]*)"'        # User-Agent  
)
```

Тут використано іменовані групи (?P<name>) для зручного доступу до елементів.

Парсінг логів в Python

```
ips = Counter()
statuses = Counter()
agents = Counter()

with open("/var/log/nginx/access.log") as f:
    for line in f:
        match = log_pattern.match(line)
        if match:
            log = match.groupdict()
            ips[log["ip"]] += 1
            statuses[log["status"]] += 1
            agents[log["user_agent"]] += 1

for ip, count in ips.most_common(10):
    print(f"{ip}: {count}")

for status, count in statuses.items():
    print(f"{status}: {count}")

for ua, count in agents.most_common(10):
    print(f"{ua}: {count}")
```

Формування HTML

```
def generate_html(ips, agents, statuses,
output_file):
    with open(output_file, "w") as f:
        f.write("<html><head><title>Log Report</
title></head><body>")
        f.write("<h1>Log analysis </h1>")

        f.write("<h2>Top IPs</h2><ul>")
        for ip, count in ips.most_common(10):
            f.write(f"<li>{ip} — {count}</li>")
        f.write("</ul>")
```

```
f.write("<h2>Top User-Agent</h2><ul>")
for ua, count in agents.most_common(10):
    f.write(f"<li>{ua} — {count}</li>")
f.write("</ul>")

f.write("<h2>HTTP statuses</h2><ul>")
for status, count in statuses.items():
    f.write(f"<li>{status} — {count}</li>")
f.write("</ul>")

f.write("</body></html>")
```

Використання argparse для параметрів командного рядка

- Бібліотека argparse дозволяє створювати CLI-інтерфейси.
- Для нашого завдання приклад такий:
- `import argparse`
- `parser = argparse.ArgumentParser(description="Log analyzer for Apache/Nginx")`
- `parser.add_argument("-f", "--file", required=True, help="Path to access.log")`
- `parser.add_argument("-o", "--output", default="report.html", help="HTML report file")`
- `parser.add_argument("--top", type=int, default=10, help="Number of top IPs/User-Agents to show")`
- `args = parser.parse_args()`
- `# Використання аргументів у коді`
- `log_file = args.file`
- Запуск:
- `python log_parser.py -f /var/log/nginx/access.log -o report.html --top 20`

Http-запити

У Python найзручніше через бібліотеку requests:

```
import requests
import time
```

```
url = "https://api.example.com/health"
start = time.time()
try:
    response = requests.get(url, timeout=5) #
timeout у секундах
```

```
response_time = time.time() - start
status_code = response.status_code
except
requests.exceptions.RequestException:
    response_time = None
    status_code = None
status_code → HTTP код відповіді (200,
404, 500 тощо)
response_time → час відповіді у секундах
При помилці RequestException endpoint
недоступний
```

Збереження результатів у JSON/CSV

JSON: зручно зберігати словники з інформацією про кожен endpoint.

```
result = {  
    "url": url,  
    "status_code": status_code,  
    "response_time": response_time,  
    "timestamp": time.strftime("%Y-%m-%d %H:  
%M:%S")  
}
```

```
import json  
with open("results.json", "a") as f:
```

```
    json.dump(result, f)  
    f.write("\n")
```

CSV: підходить для таблиць та Excel-аналізу.

```
import csv  
  
with open("results.csv", "a", newline="") as f:  
    writer = csv.writer(f)  
    writer.writerow([result["timestamp"], url,  
status_code, response_time])
```

Інтеграція з Slack/Discord для сповіщень

Slack: використовують Incoming Webhook URL

```
import requests
```

```
slack_webhook = "https://hooks.slack.com/services/XXXXX/XXXXX/XXXXX"
```

```
message = f"ALERT: {url} is down! Status code: {status_code}"
```

```
requests.post(slack_webhook, json={"text": message})
```

Discord: також через Webhook URL

```
discord_webhook = "https://discord.com/api/webhooks/XXXXX/XXXXX"
```

```
requests.post(discord_webhook, json={"content": message})
```

Повідомлення надсилаються лише при збої (status_code != 200 або timeout).

Частина 3

- Розробити CLI інструмент на Go для автоматичної організації файлів з сортуванням по типах у відповідні папки та підтримкою різних режимів роботи через command-line flags

Ідея

Розширення файлу визначає його тип:

Тип файлу Приклади розширень

Images .jpg, .jpeg, .png, .gif

Documents .pdf, .docx, .txt

Videos .mp4, .avi, .mov

Audio .mp3, .wav, .flac

Archives .zip, .tar, .rar

Для кожного типу створюється відповідна папка у цільовій директорії.

Файли переміщуються за допомогою os.Rename або копіюються з видаленням оригіналу.

```
package main

import (
    "fmt"
    "io/fs"
    "log"
    "os"
    "path/filepath"
)

func main() {
    sourceDir := "./source"
    targetDir := "./documents"

    err := os.MkdirAll(targetDir, os.ModePerm)
    if err != nil {
        log.Fatal(err)
    }

    err = filepath.WalkDir(sourceDir, func(path string, d fs.DirEntry, err
error) error {
        if err != nil {
            return err
        }
    })
}
```

```
    }

    if d.IsDir() {
        return nil
    }

    ext := filepath.Ext(d.Name())

    if ext == ".txt" || ext == ".doc" || ext == ".docx" {
        targetPath := filepath.Join(targetDir, d.Name())
        err := os.Rename(path, targetPath)
        if err != nil {
            log.Println("Error moving file:", err)
        }
    }
    return nil
})

if err != nil {
    log.Fatal(err)
}

}
```

Використання Go для CLI

flag пакет — стандартний спосіб обробки командного рядка.

Можна задавати директорію для сортування, режим роботи, фільтри типів тощо.

```
import (  
    "flag"  
    "fmt"  
)  
  
func main() {  
    dir := flag.String("dir", ".", "Directory to organize")
```

```
    mode := flag.String("mode", "once", "Mode of  
operation: once / watch")  
    dryRun := flag.Bool("dry", false, "Dry run without  
moving files")
```

```
    flag.Parse()  
  
    fmt.Println("Directory:", *dir)  
    fmt.Println("Mode:", *mode)  
    fmt.Println("Dry run:", *dryRun)  
}
```

Режими роботи (flag mode)

Одноразова організація (once):

Пробігає всі файли директорії
один раз і сортує їх.

Моніторинг (watch):

Перевіряє директорію періодично
або за подією файлової системи
(github.com/fsnotify/fsnotify пакет).

Кожного разу нові файли
переміщуються автоматично.

Dry-run режим (dry):

Симулює переміщення без
фактичної зміни структури.
Зручно для тестування.

```

func waitForCompleteFile(path string, checkInterval time.Duration, attempts int) error {
    var prevSize int64 = -1
    for i := 0; i < attempts; i++ {
        info, err := os.Stat(path)
        if err != nil {
            return err
        }
        size := info.Size()
        if size == prevSize {
            return nil
        }
        prevSize = size
        time.Sleep(checkInterval)
    }
    return fmt.Errorf("file %s is still changing", path)
}

```

```

func moveFile(path string, targetDir string) {
    ext := filepath.Ext(path)
    if ext == ".txt" || ext == ".doc" || ext == ".docx" {
        if err := waitForCompleteFile(path, 500*time.Millisecond, 5); err != nil {
            log.Println("Skipping file, still being written:", path)
            return
        }

        targetPath := filepath.Join(targetDir, filepath.Base(path))
        fmt.Printf("Moving %s -> %s\n", path, targetPath)
        err := os.Rename(path, targetPath)
        if err != nil {
            log.Println("Error moving file:", err)
        }
    }
}

```

```

func main() {
    sourceDir := "./source"
    targetDir := "./documents"

```

```

    if err := os.MkdirAll(targetDir, os.ModePerm); err != nil {
        log.Fatal(err)
    }

```

```

    watcher, err := fsnotify.NewWatcher()
    if err != nil {
        log.Fatal(err)
    }
    defer watcher.Close()

```

```

    go func() {
        for {
            select {
            case event, ok := <-watcher.Events:
                if !ok {
                    return
                }
                if event.Op&fsnotify.Create == fsnotify.Create || event.Op&fsnotify.Write == fsnotify.Write {
                    moveFile(event.Name, targetDir)
                }

            case err, ok := <-watcher.Errors:
                if !ok {
                    return
                }
                log.Println("Watcher error:", err)
            }
        }
    }()

```

```

    if err := watcher.Add(sourceDir); err != nil {
        log.Fatal(err)
    }

```

```

    fmt.Println("Watching directory:", sourceDir)
    select {}
}

```

Частина 4

- Розробити CLI інструмент на Go для автоматичної організації файлів з сортуванням по типах у відповідні папки та підтримкою різних режимів роботи через command-line flags

Git Hooks

- Git hooks — це спеціальні скрипти, які автоматично виконуються при певних діях у Git (commit, push, merge, rebase тощо).
- Вони знаходяться у .git/hooks у кожному репозиторії.
- За замовчуванням ці файли мають приклади (з розширенням .sample). Щоб увімкнути hook, потрібно створити виконуваний файл без .sample.
- Основні види hooks
 - pre-commit
 - Виконується перед створенням коміту.
 - Використовується для запуску linting/formatting.
 - Якщо повертає ненульовий код → коміт не створюється.
 - pre-push
 - Виконується перед git push.
 - Зручно для запуску тестів, щоб не відправити код із помилками.
 - Якщо повертає ненульовий код → пуш зупиняється.
 - post-commit
 - Виконується після успішного коміту.
 - Можна використовувати для backup (копіювання у резервну папку чи архів).
 - Не впливає на результат коміту (навіть якщо скрипт завершиться з помилкою).

Приклади

Pre-commit (linting)

```
#!/bin/sh
echo "Running linter..."
golangci-lint run ./...
if [ $? -ne 0 ]; then
    echo "Linting failed! Commit aborted."
    exit 1
fi
```

Pre-push (tests)

```
#!/bin/sh
echo "Running tests before push..."
go test ./...
```

```
if [ $? -ne 0 ]; then
    echo "Tests failed! Push aborted."
    exit 1
fi
```

Post-commit (backup)

```
#!/bin/sh
echo "Creating backup..."
mkdir -p ../backup
git bundle create ../backup/repo-$(date +%Y%m%d-%H%M%S).bundle --all
echo "Backup saved."
```

Не забуваємо!!! `chmod +x .git/hooks/pre-commit`

◆ pre-commit — запуск linting (flake8 або black)

```
#!/bin/sh
echo "Running flake8 linter..."
flake8 .
if [ $? -ne 0 ]; then
    echo "Linting failed! Commit aborted."
    exit 1
fi

echo "Running black formatter check..."
black --check .
if [ $? -ne 0 ]; then
    echo "Code not formatted properly (black check failed). Commit aborted."
    exit 1
fi
```

👉 Перевіряє стиль коду за допомогою flake8 і форматування через black.

Якщо помилки є — коміт зупиняється.

◆ pre-push — запуск тестів (pytest)

```
#!/bin/sh
echo "Running tests before push..."
pytest
```

```
if [ $? -ne 0 ]; then
    echo "Tests failed! Push aborted."
    exit 1
fi
```

👉 Якщо будь-який тест падає — пуш не відбудеться.

◆ post-commit — backup репозиторію

```
#!/bin/sh
echo "Creating backup..."

BACKUP_DIR="../backup"
BACKUP_FILE="$BACKUP_DIR/repo-backup.bundle"

mkdir -p "$BACKUP_DIR"

if [ -f "$BACKUP_FILE" ]; then
    rm "$BACKUP_FILE"
    echo "Old backup removed."
fi

git bundle create "$BACKUP_FILE" --all

echo "Backup saved to $BACKUP_FILE"
```

Cron

cron — сервіс для автоматичного запуску команд та скриптів за розкладом

Використовується для регулярних завдань: резервне копіювання, очищення логів, оновлення даних

Таблиця cron (crontab)

- * * * * * command_to_run
- | | | | |
- | | | | | — день тижня (0-7, де 0 або 7 = неділя)
- | | | | | — місяць (1-12)
- | | | | | — день місяця (1-31)
- | | | | | — година (0-23)
- | | | | | — хвилина (0-59)

Приклади використання

- Щоденний запуск скрипта о 2:30 ранку
- 30 2 * * * /home/user/backup.sh

- Щогодинний запуск
- 0 * * * * /home/user/check_logs.sh

- Кожні 5 хвилин
- */5 * * * * /home/user/sync_data.sh

Основні команди

- crontab -e — редагувати власний crontab
- crontab -l — переглянути поточні завдання
- crontab -r — видалити всі завдання користувача

💡 Поради:

- Переконайтеся, що скрипт має право на виконання (chmod +x script.sh)
- Використовуйте повні шляхи до команд і файлів у cron
- Логи cron зазвичай можна перевірити у /var/log/cron або /var/log/syslog

<https://opensource.com/article/17/11/how-use-cron-linux>